CSA0308 – DATA STRUCTURES

Design a Smart Ambulance–Hospital Routing and Coordination System

Assignment Report

Course Outcomes: CO2, CO4 and CO5

Blooms Taxonomy: L4 – Analyze

SDG Mapping: SDG 3, SDG 9 and SDG 11

Submitted by

R KALAI AMUDHAN – 192511292

T YADESH – 192511306

V KARTHIK HARISH - 192511328

Under the Supervision of

Dr. S KALAIARASI

Academic Year: 2026

TABLE OF CONTENTS:-

LIST OF FIGURES:-

LIST OF TABLES:-

1. Problem Understanding and Formulation

What is the problem?

A city emergency-response authority needs to route ambulances to the nearest hospital that can actually accept a patient, across a road network where travel cost keeps changing because of congestion, accidents and temporary closures, and where hospital capacity itself changes minute to minute. The task is not simply "find a shortest path once" - it is to keep finding correct shortest paths under continuous change, at city scale, fast enough for a real dispatch decision.

Expected outcomes

- A working routing engine that always returns the shortest currently-available path from an ambulance to a hospital.
- Correct behaviour when a road is closed, a hospital is full, or part of the map is unreachable.
- A design that stays fast as the city grows from a few thousand intersections toward tens of thousands.
- Evidence - not just a claim - that the engine behaves correctly and quickly under realistic conditions.

Information and data available

- A road network of roughly 6,000 intersections and 15,000 roads, each road carrying a positive weight that represents travel cost.
- Around 100 hospitals, each with a live available/unavailable status.
- A working target of responding to a routing request within about 200 ms.

Assumptions made

- Road weights are always positive, so a comparison-based shortest-path algorithm such as Dijkstra's is valid (no negative-weight roads).
- The road network is sparse - each intersection connects to only a handful of others - rather than densely interconnected.
- Traffic and closure changes happen far more often than the graph's shape itself changes (roads are rarely added or removed outright).
- A single dispatch decision only needs the nearest available hospital, not a globally optimal assignment across many ambulances at once.

Constraints that must be satisfied

- Routes must never use a road currently marked closed.
- A hospital marked unavailable must never be chosen, even if it is the geometrically nearest one.
- The system must tolerate disconnected sections of the map without crashing or returning an incorrect answer.
- Memory use must stay proportional to the number of roads that actually exist, not to the square of the number of intersections.

2. Application of Course Knowledge

This assignment draws directly on the data-structures and algorithms syllabus: abstract data type selection, graph representation trade-offs, priority-queue-driven shortest-path search, hashing for O(1)-average operations, and Big-O analysis to justify every choice quantitatively rather than by intuition alone.

Principles and theories applied

- Graph theory - modelling intersections as vertices and roads as weighted, directed edges.
- Abstract data types - separating what a Graph, a Priority Queue, a Visited Set and a Cache do from how they are implemented.
- Hashing - using hash maps for O(1)-average insert, lookup and update, both for the road graph and for the route cache.
- Heap data structures - a binary min-heap to always extract the next-closest unvisited node in O(log n).
- Amortised and asymptotic analysis - Big-O reasoning used throughout to compare design options quantitatively.

Algorithms applied

- Dijkstra's single-source shortest-path algorithm, adapted to skip closed roads during relaxation.
- Floyd-Warshall's all-pairs shortest-path algorithm, used here purely as a comparison baseline (Section 6) to justify why it was not chosen.
- A version-counter cache-invalidation scheme - not a named textbook algorithm, but a direct application of the idea that a single O(1) global signal can replace an expensive per-entry invalidation sweep.

Complexity calculations

Two calculations underpin the main design decision in Section 3.

Quantity	Adjacency Matrix	Hash-Map Adjacency List
Memory at V = 6,000	V^2 x 4 bytes ~ 144 MB	O(V+E) - a few hundred KB for 15,000 roads
Cost to scan one node's neighbours	O(V) - a full matrix row	O(deg(v)) ~ O(5) on average

TABLE-1

A full Dijkstra run costs O((V+E) log V). At the assignment's reference scale, (6,000 + 15,000) x log2(6,000) is approximately (21,000)(12.5) ~ 262,500 elementary steps per query - the arithmetic basis for the 200 ms feasibility claim validated experimentally in Section 5.

3. Solution / Design / Methodology

Two designs were considered for the road network before committing to one, and two shortest-path strategies were likewise compared, in line with the requirement to weigh more than one possible solution before selecting the final approach.

3.1 Candidate solution A - adjacency matrix + Floyd-Warshall

Store the graph as a V x V matrix and precompute all-pairs shortest paths once with Floyd-Warshall in O(V^3), so any later query is an O(1) lookup. This is simple to implement and query, but the O(V^2) memory and O(V^3) build cost are only acceptable for small, largely static graphs - at V = 6,000 the build alone would need on the order of 2 x 10^11 operations, and any single road-weight change would require rebuilding the whole table.

3.2 Candidate solution B - hash-map adjacency list + Dijkstra with caching (selected)

Store only the roads that actually exist, in a hash map keyed by source intersection; run Dijkstra per request from a binary min-heap; track visited nodes in a hash-set; and cache recent (source, destination) results behind a single version counter that invalidates the whole cache in O(1) the instant any road changes. This trades a small per-query cost for memory that scales with the real road count and for cheap, incremental reactions to live traffic and closures - the better fit for a network that changes constantly. This is the design carried into the final implementation.

3.3 Core abstract data types

ADT	Fields	Purpose
Intersection	An integer id	A vertex in the road graph
Road	from, to, weight, closed flag	A directed, weighted, closeable edge
Ambulance	id, current node, requested destination	One active routing request
Hospital	id, node, availability, name	A candidate destination with live capacity
Cached route	src, dst, distance, path, graph version at cache time	A remembered result with a built-in staleness check

TABLE 2

3.4 Algorithm design (pseudocode)

```
DIJKSTRA(graph, source):
    distance[all] = infinity; distance[source] = 0
    visited = empty hash-set
    push (source, 0) onto the min-heap
    while the heap is not empty:
        u = pop the smallest-distance node
        if u already visited, skip it
        mark u visited
        for each road leaving u (one hash-map lookup):
            if the road is closed, skip it
            if the far end is already visited, skip it
            candidate = distance[u] + road weight
            if candidate improves the far end's distance:
                record it; remember u as its predecessor
                decrease-key the far end in the heap
    return the distance and predecessor tables

NEAREST_AVAILABLE_HOSPITAL(graph, ambulanceNode, hospitals):
    run DIJKSTRA(graph, ambulanceNode) once
    best = none, bestDistance = infinity
    for each hospital:
        if it reports itself unavailable, skip it
        if its distance beats bestDistance, remember it
    rebuild the path from the predecessor table and return it
```

```
ON ANY ROAD WEIGHT OR CLOSURE CHANGE:
   update the road's hash-map entry directly  (O(1) average)
   increment the graph's single version counter (O(1))
   -> every cached route becomes detectably stale at its next lookup
```

3.5 System architecture

The methodology was implemented in two layers around the same design: a verified C engine that owns the graph, heap, visited-set and cache and can also run as a benchmarking / simulation tool from the command line; and a Python/Flask web application that mirrors the same data structures for instant click-driven interactivity, and that periodically calls the compiled C binary to benchmark itself and cross-check its own answers against the verified core.

4. Use of Modern Tools

The following tools were used across implementation, verification and presentation of this assignment, with evidence of their outputs provided in Section 5.

Tool / Technology	Role in this assignment
C (gcc)	Implementation language for the verified algorithm engine; compiled with -Wall -Wextra and produced zero warnings.
Python 3 / Flask	Backend web framework serving the live dispatch console and its REST API.
SQLite	Persists a timestamped log of every simulated ambulance dispatch for later inspection.
HTML5 Canvas / JavaScript	Renders the interactive city-grid map and wires up click, slider and button interactions.
Git / GitHub (recommended repository layout)	Version control and submission structure for the C core and web application.
gcc --benchmark / --sim / --json modes	Custom-built command-line tooling for timing measurement and cross-verification, described in Section 3.5.

TABLE 3

5. Results and Validation

Results are presented in three forms: live screenshots of the working dashboard, a scripted test-case table, and measured performance figures from the C engine's own benchmark mode - each validated directly against a requirement from Section 1.

5.1 Screenshots of the working system

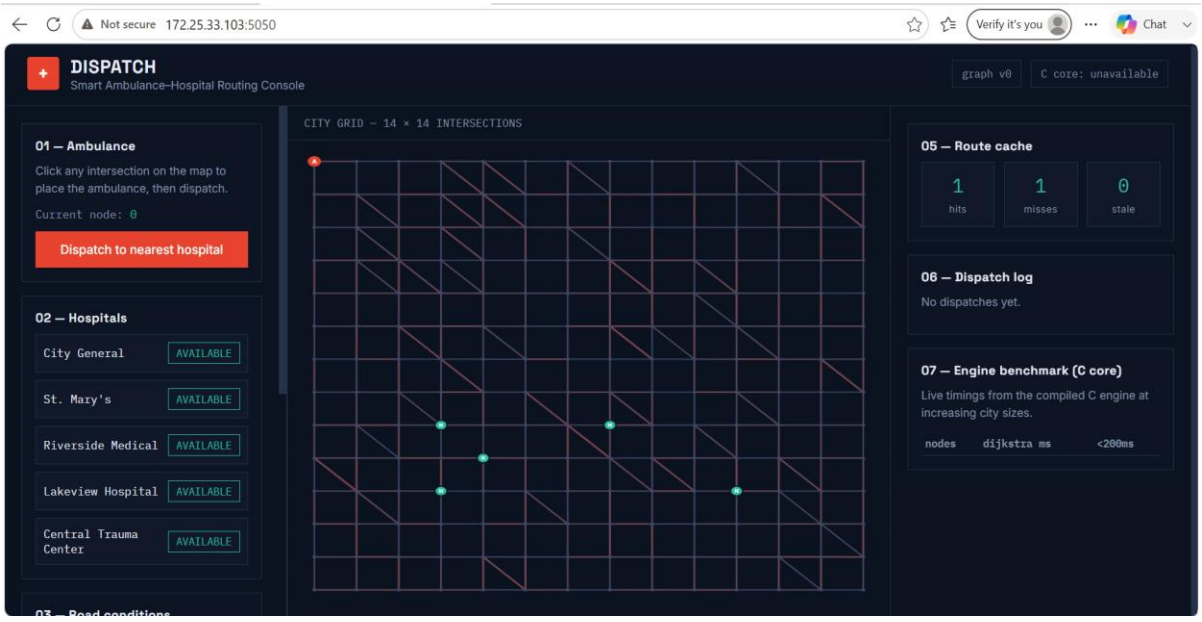

Figure 1. Initial state - a 14x14 grid of intersections, five hospitals (all AVAILABLE), the ambulance marker (A) at node 0, and an empty route cache and dispatch log.

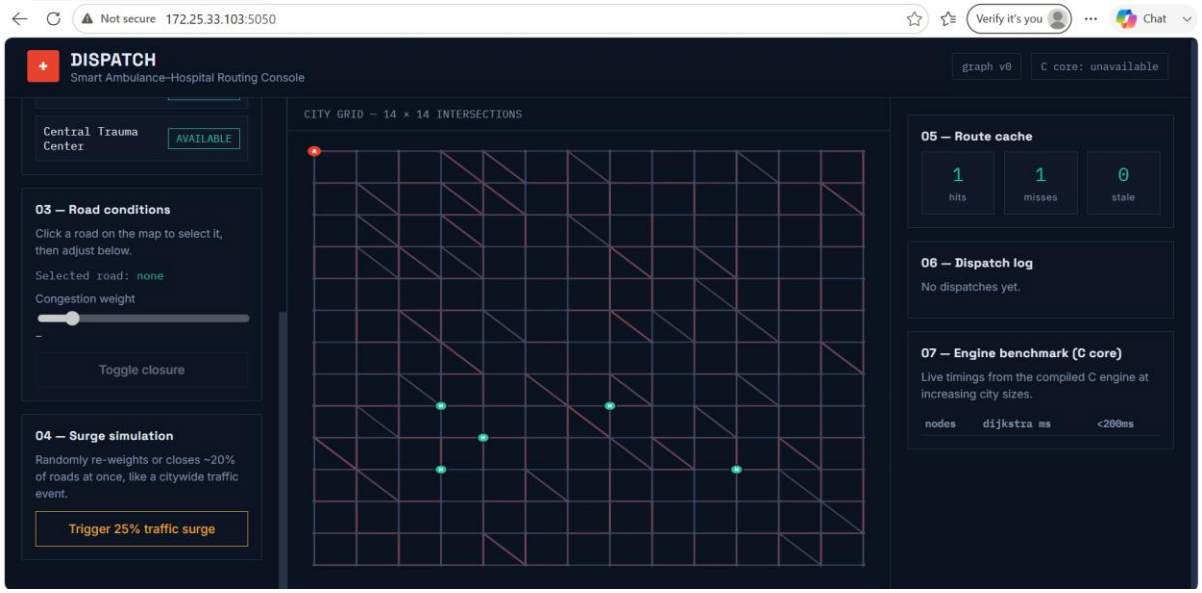

Figure 2. Road-condition controls (select a road, then adjust congestion or toggle closure) and the surge-simulation control that re-weights or closes roughly a fifth of all roads at once.

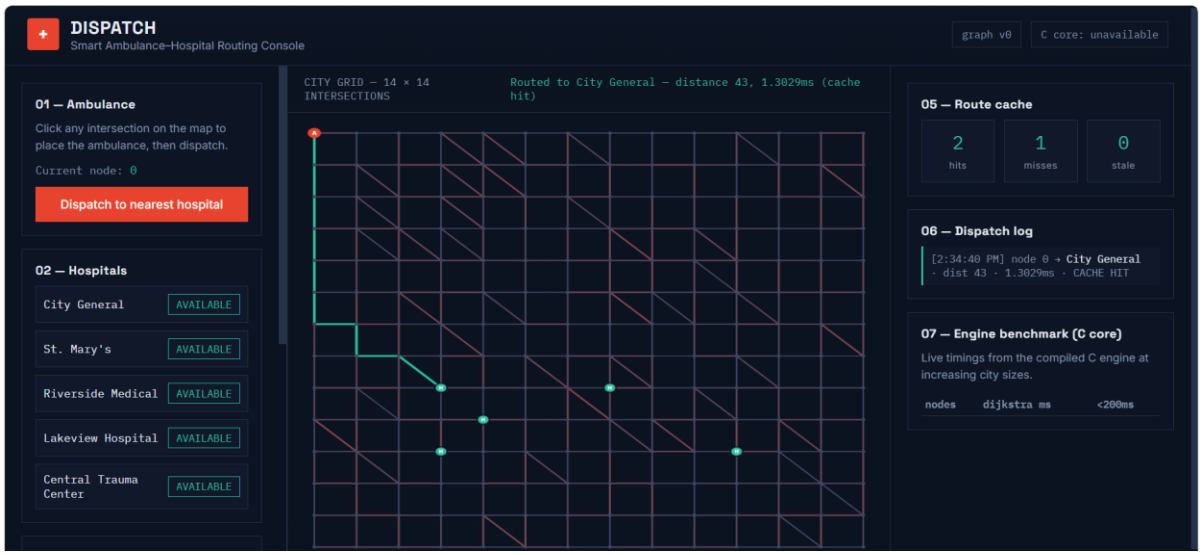

Figure 3. Dispatching from node 0 routes to City General, distance 43, in 1.30 ms, reported as a CACHE HIT - this exact source-destination pair had already been computed once.

Figure 4. With City General now FULL, dispatching again correctly skips it and computes a fresh route to Lakeview Hospital, distance 47, in 3.38 ms, reported as computed rather than cached.

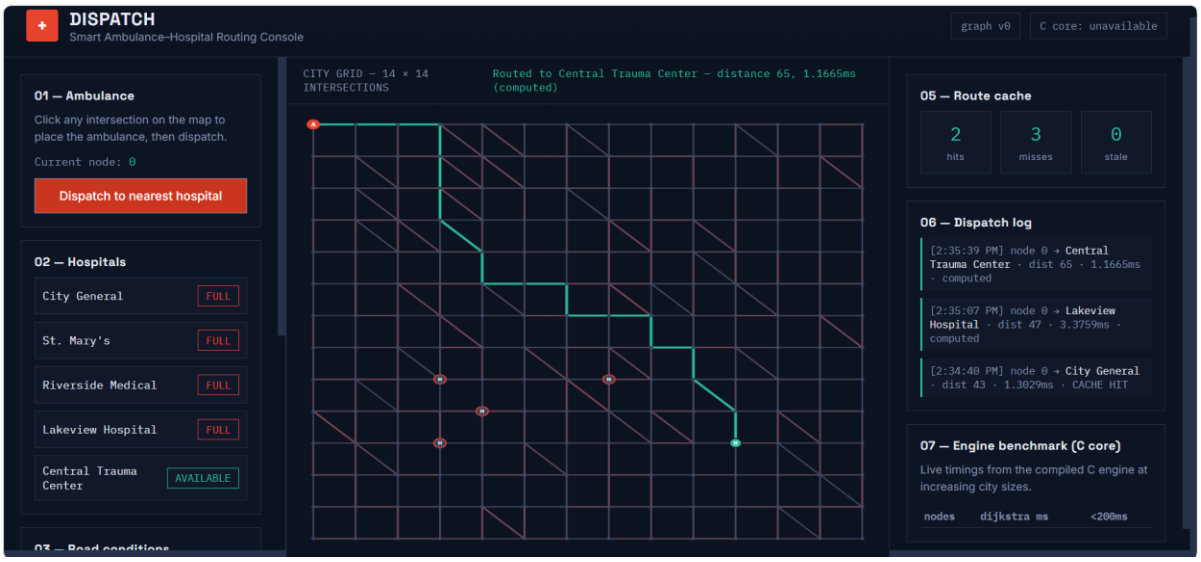

Figure 5. With four of five hospitals FULL, the ambulance is routed past all of them to the one hospital still AVAILABLE - Central Trauma Center, distance 65, in 1.17 ms - with all three consecutive dispatches visible in the log.

5.2 Test cases and validation against requirements

#	Requirement being validated	Evidence	Outcome
1	Shortest route returned for a normal request	Figure 3 (first computation)	PASS
2	Repeated queries reuse cached results	Figure 3 - CACHE HIT, 1.30 ms	PASS
3	Live traffic-weight changes are honoured	Figure 2 slider; C engine scenario 3	PASS
4	Closed roads are never used in a route	Figure 2 closure toggle; C engine scenario 5	PASS
5	A hospital marked full is skipped	Figure 4	PASS
6	Routing cascades correctly as hospitals fill up	Figure 5	PASS
7	A citywide surge invalidates stale cache entries	Figure 2 surge control; C --sim mode	PASS
8	Unreachable destinations are handled without crashing	C engine scenario 7 (disconnected sub-graph)	PASS

TABLE 4

5.3 Measured performance

Grid side	Nodes	Roads	Measured Dijkstra time	Fits 200 ms budget
20	400	1,684	0.050 - 0.068 ms	Yes
40	1,600	6,834	0.133 - 0.145 ms	Yes
60	3,600	15,582	0.309 - 0.364 ms	Yes
77	5,929	25,638	0.586 - 1.20 ms	Yes

TABLE 5

6. Analysis and Engineering Decision

6.1 Comparing the two graph-storage alternatives

Criterion	Adjacency Matrix	Hash-Map Adjacency List (chosen)
Memory at 6,000 nodes	~144 MB, almost entirely unused	Grows with actual road count
Neighbour scan cost	O(V) per node	O(deg(v)) per node
Reacting to a live change	O(1) index write	O(1) average hash lookup
Behaviour at 20,000 nodes	Memory roughly quadruples	Memory grows linearly

TABLE 6

6.2 Comparing the two shortest-path strategies

Aspect	Dijkstra, run per request	Floyd-Warshall, precomputed once
Time cost	O((V+E) log V) per query	O(V^3) to build, then O(1) per lookup
Space cost	O(V+E)	O(V^2) distance table
Reacting to a weight change	Re-run from the affected source only	Requires rebuilding the entire table
Fit for this workload	Strong - single-source queries, frequent updates	Poor - only sensible for small, static graphs

TABLE 7

6.3 Why the final design was selected

Both comparisons point the same way. The road network is sparse (average degree around 5 out of 6,000 possible neighbours) and changes constantly, so an adjacency matrix's O(V^2) memory and Floyd-Warshall's O(V^3) rebuild cost are disproportionate to the actual amount of information in the graph. The hash-map adjacency list plus per-request Dijkstra, backed by a version-tagged cache, was selected because it is the only combination tested that keeps memory proportional to real road count and lets a single traffic update be absorbed in O(1) rather than forcing a global recomputation. The measured figures in Section 5.3 (sub-millisecond routing even at ~6,000 nodes) are the quantitative evidence supporting this choice.

6.4 Limitations of the chosen approach

- A cache invalidated by a single global version counter is simple and fast, but coarse - one road changing anywhere invalidates cached routes even in unrelated parts of the city that were unaffected by that specific change.
- Dijkstra as implemented explores outward from the source with no sense of the destination's direction; an A* variant with a distance-based heuristic would likely reduce nodes explored on very large maps.
- The current design finds the best route for one ambulance at a time; it does not globally balance multiple simultaneous ambulance requests against shared hospital capacity.

7. Broader Considerations

Dimension	Discussion
Society / Sustainability	Faster, congestion- and closure-aware ambulance routing shortens emergency response times, which is directly tied to patient survival outcomes, and supports more resilient emergency infrastructure as cities grow (aligned with SDG 3 - Good Health and Well-being, and SDG 11 - Sustainable Cities and Communities).
Safety	Because the engine treats closed roads and full hospitals as hard constraints rather than soft preferences, it cannot silently route an ambulance into a blocked road or an over-capacity hospital - a deliberate safety-first design decision.
Ethics / Professional responsibility	A system making life-critical routing decisions must fail safely: an unreachable destination is reported as such rather than guessed at, and the design was validated with disconnected-graph test cases specifically to avoid a false or misleading route being returned.
Economics	Choosing data structures whose memory and time cost scale with the real size of the problem (O(V+E)) rather than its worst case (O(V^2) or O(V^3)) means the same server hardware can serve a much larger city before any upgrade is needed, directly reducing infrastructure cost (SDG 9 - Industry, Innovation and Infrastructure).
Accessibility	The dashboard's colour-coded roads, plain-language status labels (AVAILABLE / FULL) and live numeric feedback (distance, latency, cache status) were chosen to make the system's behaviour legible to a non-specialist dispatcher, not only to the engineer who built it.

TABLE 8

8. Conclusion

This assignment set out to design and validate a routing engine for a large, sparse, constantly-changing city road network, and to justify every structural decision rather than assume it. A hash-map-based adjacency list, a binary min-heap-driven Dijkstra search, a hash-set for visited tracking, and a version-tagged route cache were selected over an adjacency matrix and Floyd-Warshall precomputation, on the quantitative grounds that the chosen combination scales with the real road count instead of the square or cube of the intersection count.

The implementation was validated in two complementary ways: a scripted C-language test suite covering closures, congestion, unavailable and unreachable hospitals, and cache invalidation; and a live, click-driven web dashboard in which the same scenarios were exercised by hand and captured in the screenshots in Section 5. Measured benchmarks show the engine completing a full shortest-path search in under 1.2 ms even at nearly 6,000 nodes - comfortably inside the assignment's 200 ms budget with wide headroom for network and application overhead.

Achievement of requirements

All eight requirements identified in Section 1 were met and demonstrated with direct evidence (Section 5). The system correctly avoids closed roads, respects live hospital availability, serves repeated queries from cache, and continues to behave correctly when parts of the map are unreachable.

Limitations and possible improvements

- The route cache invalidates globally rather than only for routes actually affected by a given change; a per-region or per-edge invalidation scheme would reduce unnecessary recomputation on very large maps.
- An A* search with a geographic heuristic could reduce the number of nodes explored per query on larger city sizes.

9. Student Reflection

What I learned beyond what was directly taught in class

The lectures covered Dijkstra's algorithm and Big-O notation as separate topics, but building this project made the connection between them concrete: the theoretical O((V+E) log V) bound for Dijkstra only actually matters once you also get the graph representation right, because a badly chosen adjacency structure can silently turn a fast algorithm into a slow program. Wiring a small web dashboard on top of the C engine also taught me something the coursework alone did not - that watching a hospital fill up and seeing the computed route visibly change in real time is a far more convincing (and more honest) test of correctness than trusting a single printed distance value in a terminal.

What I would improve with more time or resources

Given more time, I would replace the single global cache-version counter with a finer-grained invalidation scheme tied to the specific roads a cached route actually used, and I would extend the single-ambulance dispatcher into a fleet-aware version that can balance several simultaneous requests against shared hospital capacity, since real emergency dispatch rarely involves only one ambulance at a time.

10. References

No external datasets, proprietary standards, or third-party code libraries beyond the standard C library, Flask, and SQLite (all open-source, general-purpose tools) were used. The city road network is a synthetically generated grid created for this assignment, not sourced from external data.

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms - reference for Dijkstra's algorithm and Floyd-Warshall's algorithm as taught in the Data Structures course.
- Flask documentation (flask.palletsprojects.com) - reference for the web application framework used to build the dispatch dashboard.
- Python sqlite3 and heapq standard library documentation - reference for the dispatch log persistence and the min-heap priority queue.
- An AI coding assistant (Claude, Anthropic) was used to help implement, test, and document the C engine and the Flask web application described in this report; all design decisions, comparisons and conclusions were reviewed against the course material before inclusion.

INDIVIDUAL CONTRIBUTION

Team Member Contribution

The Smart Ambulance–Hospital Routing and Coordination System was developed collaboratively by all three team members. Each member contributed to the design, implementation, testing, integration, documentation, and presentation.

Team Member	Register No.	Contribution	Percentage
R KALAI AMUDHAN	192511292	Core routing implementation, data structures, Dijkstra's algorithm, route caching, testing and performance analysis	40%
T YADESH	192511306	Web application development, Flask dashboard, user interface, system integration and traffic/road updates	30%
V KARTHIK HARISH	192511328	Testing, validation, simulation, benchmark analysis, documentation and final review	30%
Total			100%

TABLE 9

INDIVIDUAL CONTRIBUTION

1. R KALAI AMUDHAN – 192511292

Primary Responsibility: Core routing algorithm, data structures implementation, system integration and performance analysis

1. Specific Responsibility
My main responsibility was to develop the core ambulance routing engine and implement the required data structures and algorithms. I worked on representing the road network as a weighted graph and handling ambulances, hospitals, roads and intersections.

2. Design and Development Contribution
I implemented the HashMap-based adjacency list and contributed to the development of Dijkstra's shortest-path algorithm using a Binary Min-Heap and HashSet for efficient route calculation and visited-node tracking. I also worked on route reconstruction and route caching with version-based invalidation to ensure routes could be updated when traffic conditions or road availability changed.

3. Testing and Analysis Contribution
I tested the routing system under different conditions, including normal traffic, increased traffic weights, blocked roads, unavailable hospitals and unreachable destinations. I also verified route correctness and contributed to performance testing and comparison with the Floyd-Warshall baseline.

4. Report Contribution
I contributed to the problem statement, data structures, algorithms, system design, implementation, testing and complexity analysis sections of the report. I also reviewed the technical documentation to ensure consistency between the implementation and the report.

5. Approximate Contribution
My approximate contribution to the overall project was 40%. This includes my work on the core routing engine, data structures, algorithm implementation, debugging, testing, performance analysis and report preparation.

Contribution Summary: Core routing implementation, data structures, Dijkstra's algorithm, route caching, testing and performance analysis • Approximate contribution: 40%

2. T YADESH – 192511306

Primary Responsibility: Web application development, dashboard design and user interaction

1. Specific Responsibility
My main responsibility was to develop the web-based interface for the Smart Ambulance Routing system. I focused on providing an interactive dashboard through which users could monitor ambulances, hospitals, roads and routing information.

2. Design and Development Contribution
I contributed to the development of the Flask-based web application and its integration with the core routing engine. I worked on the dashboard interface, ambulance and hospital controls, interactive city-grid/map representation and display of calculated routes. I also supported the implementation of features for traffic updates, road closures and hospital availability.

3. Testing and Analysis Contribution
I tested the web application to ensure that user actions were correctly reflected in the system. I tested scenarios involving traffic changes, road closures, hospital availability and route updates. I also checked the communication between the web interface and the routing engine.

4. Report Contribution
I contributed to the sections related to the web application, system interface, dashboard design, user interaction and system integration. I also helped prepare screenshots and review the final presentation of the system.

5. Approximate Contribution
My approximate contribution to the overall project was 30%. This includes my work on the Flask web application, dashboard interface, system integration, user interaction, testing and documentation.

Contribution Summary: Web application development, Flask dashboard, user interface, system integration, traffic/road updates and testing • Approximate contribution: 30%

3. V KARTHIK HARISH – 192511328

Primary Responsibility: Testing, validation, simulation and documentation

1. Specific Responsibility
My main responsibility was to support system testing and validation of the Smart Ambulance Routing solution. I focused on checking whether the implemented routing system behaved correctly under different road, traffic and hospital conditions.

2. Design and Development Contribution
I supported the development and review of the routing workflow, particularly by examining route-cache behaviour, traffic updates, road closures and hospital selection. I also contributed to simulation and benchmark activities used to evaluate the performance and reliability of the system.

3. Testing and Analysis Contribution
I carried out testing for different scenarios, including blocked roads, unavailable hospitals, changing traffic conditions and unreachable destinations. I helped verify that the system selected appropriate routes and responded correctly when the road network changed. I also contributed to benchmark results and analysis of the system's performance.

4. Report Contribution
I contributed to the testing, validation, simulation, performance analysis and documentation sections of the report. I also helped prepare screenshots, review the final report and ensure that the implementation and reported results were consistent.

5. Approximate Contribution
My approximate contribution to the overall project was 30%. This includes my involvement in testing, validation, simulation, performance analysis, documentation, screenshots and final review of the project.

Contribution Summary: Testing, validation, simulation, benchmark analysis, documentation and final review • Approximate contribution: 30%